LAB5 - Copy On Write

Srikarthik Kanithi
Roll No: CS23B029

1 Changes I Made

1.1 Makefile

Added the following in the `UPROGS` section:

```
$U/_cowtest\
```

1.2 kalloc.c

At the start of the file:

```
struct spinlock ref_lock;
uint16 page_ref[PHYSTOP/PGSIZE];
```

In the `kinit` function (before `freerange`):

```
initlock(&ref_lock, "refcnt");
for(int i = 0; i < PHYSTOP/PGSIZE; i++)
    page_ref[i] = 0;
```

In the `kfree` function (before filling with junk):

```
uint64 index = PA2IDX((uint64)pa);
acquire(&ref_lock);
if(page_ref[index] > 0)
    page_ref[index]--;
if(page_ref[index] > 0){
    release(&ref_lock);
    return;
}
release(&ref_lock);
```

In the `kalloc` function (after filling with junk):

```
acquire(&ref_lock);
page_ref[PA2IDX((uint64)r)] = 1;
release(&ref_lock);
```

1.3 memlayout.h

Added at the end:

```
#define PGSIZE 4096
#define PA2IDX(pa) ((pa) / PGSIZE)
```

1.4 vm.c

In `uvmunmap` (inside `if(do_free)`):

```
if(do_free){
    uint64 pa = PTE2PA(*pte);
    acquire(&ref_lock);
    if(page_ref[PA2IDX(pa)] > 0){
        page_ref[PA2IDX(pa)]--;
        if(page_ref[PA2IDX(pa)] == 0){
            release(&ref_lock);
            kfree((void*)pa);
            goto skip;
        }
    }
    release(&ref_lock);
}
skip:
*pte = 0;
```

In `uvmcopy` (replace flags code):

```
if((flags & PTE_W)){
    *pte = PA2PTE(pa) | (flags & ~PTE_W) | PTE_COW | PTE_V | (
       flags & PTE_U);

    if(mappages(new, i, PGSIZE, pa, (flags & ~PTE_W) | PTE_COW |
       PTE_V) != 0)
        goto err;

    acquire(&ref_lock);
    page_ref[PA2IDX(pa)]++;
    release(&ref_lock);
} else {
    if(mappages(new, i, PGSIZE, pa, flags) != 0)
        goto err;

    acquire(&ref_lock);
    page_ref[PA2IDX(pa)]++;
    release(&ref_lock);
}
```

In `copyout` (replace while loop):

```
while(len > 0){
    va0 = PGROUNDDOWN(dstva);
    if(va0 >= MAXVA)
```

```
        return -1;
    pte = walk(pagetable, va0, 0);
    if(pte == 0 || (*pte & PTE_V) == 0 || (*pte & PTE_U) == 0)
        return -1;

    if(((*pte) & PTE_W) == 0){
        if((*pte & PTE_COW) == 0)
            return -1;

        uint64 old_pa = PTE2PA(*pte);
        char *mem = 0;
        int do_copy = 0;

        acquire(&ref_lock);
        if(page_ref[PA2IDX(old_pa)] > 1){
            page_ref[PA2IDX(old_pa)]--;
            do_copy = 1;
        }
        release(&ref_lock);

        if(do_copy){
            mem = kalloc();
            if(mem == 0)
                return -1;
            memmove(mem, (char*)old_pa, PGSIZE);
            acquire(&ref_lock);
            page_ref[PA2IDX((uint64)mem)] = 1;
            release(&ref_lock);

            uint64 newflags = (PTE_FLAGS(*pte) & ~PTE_COW) |
               PTE_W;
            *pte = PA2PTE((uint64)mem) | newflags | PTE_V;
            sfence_vma();
        } else {
            uint64 newflags = (PTE_FLAGS(*pte) & ~PTE_COW) |
               PTE_W;
            *pte = PA2PTE(old_pa) | newflags | PTE_V;
            sfence_vma();
        }
    }
    pa0 = PTE2PA(*pte);
    n = PGSIZE - (dstva - va0);
    if(n > len)
        n = len;
    memmove((void *)(pa0 + (dstva - va0)), src, n);

    len -= n;
    src += n;
    dstva = va0 + PGSIZE;
}
```

1.5 riscv.h

Added:

```
#define PTE_COW (1L << 8)
```

1.6 trap.c

Added in usertrap:

```
else if(r_scause() == 15){
    uint64 va = r_stval();
    if(va >= MAXVA){
        printf("usertrap(): page fault va %p\n", va);
        setkilled(p);
        goto out;
    }
    uint64 va0 = PGROUNDDOWN(va);
    pte_t *pte = walk(p->pagetable, va0, 0);
    if(pte == 0 || ((*pte & PTE_V) == 0) || ((*pte & PTE_U) == 0)
       ){
        printf("usertrap(): page fault va %p\n", va);
        setkilled(p);
        goto out;
    }
    if(((*pte & PTE_COW) != 0) && (((*pte & PTE_W) == 0))){
        uint64 old_pa = PTE2PA(*pte);
        char *mem = 0;
        int do_copy = 0;
        acquire(&ref_lock);
        if(page_ref[PA2IDX(old_pa)] > 1){
            page_ref[PA2IDX(old_pa)]--;
            do_copy = 1;
        }
        release(&ref_lock);
        if(do_copy){
            mem = kalloc();
            if(mem == 0){
                setkilled(p);
                goto out;
            }
            memmove(mem, (char*)old_pa, PGSIZE);
            acquire(&ref_lock);
            page_ref[PA2IDX((uint64)mem)] = 1;
            release(&ref_lock);

            uint64 newflags = (PTE_FLAGS(*pte) & ~PTE_COW) |
               PTE_W;
            *pte = PA2PTE((uint64)mem) | newflags | PTE_V;
            sfence_vma();
        } else {
```

```
            uint64 newflags = (PTE_FLAGS(*pte) & ~PTE_COW) |
               PTE_W;
            *pte = PA2PTE(old_pa) | newflags | PTE_V;
            sfence_vma();
        }
    } else {
        printf("usertrap(): page fault va %p\n", va);
        setkilled(p);
        goto out;
    }
}
```

1.7 defs.h

Added at the start of the file:

```
extern struct spinlock ref_lock;
extern uint16 page_ref[];
```

2 Brief Code Explanation

- Normally, without COW, the `uvmcopy` function in `fork` creates a new user address space for the child by allocating a new leaf PTE for every VA and copying the parent's physical page into the new page. This consumes a large amount of memory.
- To avoid this, we use **Copy-On-Write (COW)**. Instead of creating a new page, both parent and child PTEs point to the same physical page. The write bit is disabled, and the COW bit is set. If the original parent PTE did not have write enabled, then no COW is applied.
- After forking, both parent and child page tables contain some pages as COW and some pages as just read-only.
- A global reference counter array tracks how many processes are using each physical page.
- On `kalloc`, the reference count is initialized to 1 (one process using that page).
- On `kfree`, before freeing, we check if any process is still using the page. Only when the reference count becomes 0 do we clear that page's memory.
- Since the reference counter is shared across processes, we use **spinlocks** to prevent race conditions when accessing or modifying it.
- The `uvmunmap` function in `uvmcopy` works similarly to `kfree`, decrementing reference counts and freeing pages when safe.
- The `copyout` function copies kernel memory into user memory by walking the page table. It checks each page:

  - If the leaf PTE is read-only and has COW set: allocate a new page, copy the old page into it, update reference counts, and update the PTE with write enabled.
  - If COW is not set and write is disabled: return error.
  - If write is enabled: directly copy data into the page.

- The `usertrap` logic for handling page faults is very similar to `copyout`.